PROJET FIL ROUGE CI/CD · COMPTE RENDU

Journée 1 – Intégration continue

SouverainRecrut Backend : tests, SonarQube, Jenkins, Docker, Trivy, Docker Hub

Projet	SouverainRecrut – API Backend (Node.js 22, Express, TypeScript, Prisma, PostgreSQL)
Module	CI/CD – Mastère Expert en Développement – RNCP 40166 · Séance 1 (cahier des charges, partie 3)
Dépôt	https://github.com/mtbvr/CICD_souverainrecrut – branche `dev`
Auteur	Matéo Bouvier – 9 novembre 2026

1 Objectif et résultat
2 Architecture et lab
3 Tests et couverture
4 SonarQube
5 Image Docker et Trivy
6 Pipeline Jenkins
7 Difficultés

1. Objectif et résultat

Chaque modification du Backend sur la branche `dev` est automatiquement compilée, testée, analysée par SonarQube, conteneurisée et scannée par Trivy, puis **publiée sur Docker Hub uniquement si tous les contrôles sont passés**.

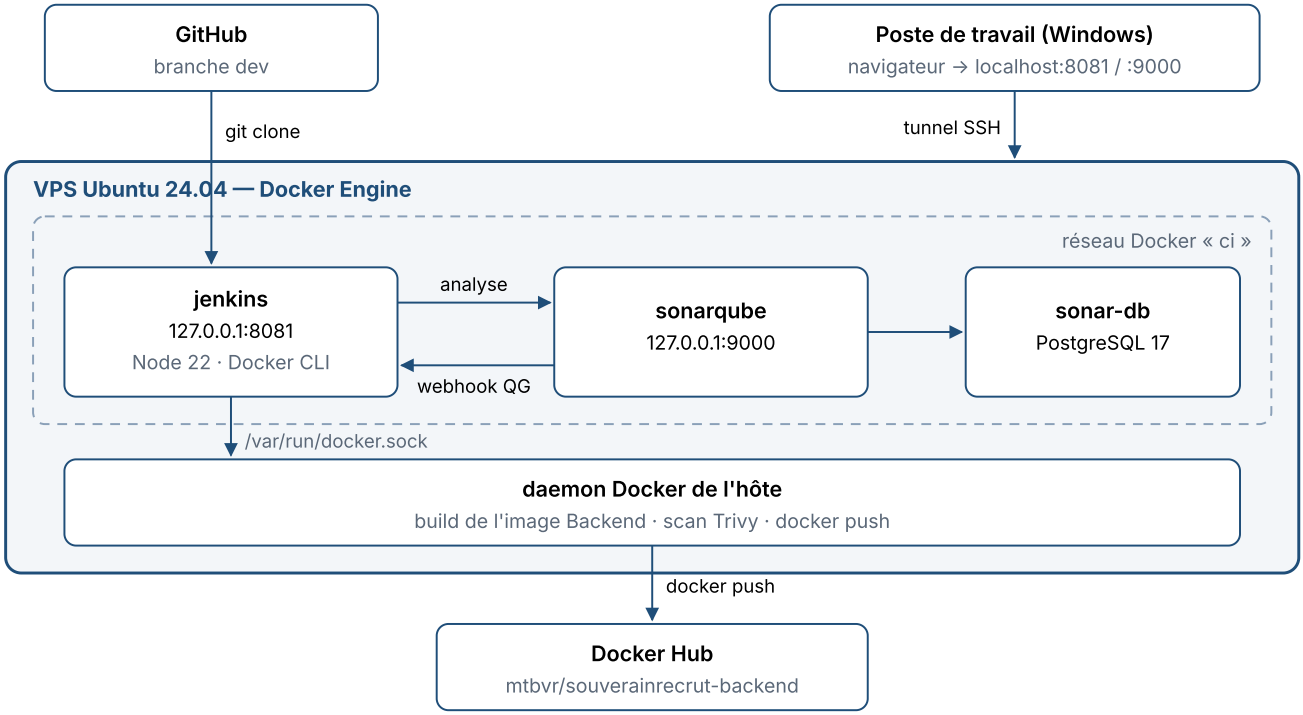
Le pipeline est fonctionnel, et les builds de la journée montrent que chaque contrôle bloque réellement la publication :

Build	Tag de l'image	Résultat	Cause
#1	`1.0.0-61c559b`	Bloqué par Trivy	CVE critique dans une dépendance (section 5)
#2	`1.0.0-563928b`	Publié	CVE corrigée
#3	`1.0.0-563928b`	Bloqué par le Quality Gate	Gate personnalisé plus strict (section 4)
#4	`1.0.0-172892d`	Publié	Code corrigé
#5	`1.0.0-8b039e7`	Publié	Image durcie : **version de référence**



Stage View Jenkins des builds #1 à #4

2. Architecture et lab



Jenkins, SonarQube et sa base PostgreSQL tournent en Docker Compose sur un VPS. Jenkins utilise le Docker de l'hôte pour construire l'image et lancer Trivy. Les deux interfaces n'écoutent que sur `127.0.0.1` et sont accessibles depuis le poste par tunnel SSH.

Le lab s'installe et se vérifie avec trois scripts : `setup-vm.sh`, `start-lab.sh` et `check-lab.sh`.

```
ubuntu@vps-841b4090:~/CICD_souverainrecrut/integration-continue/lab$ ./scripts/check-lab.sh
== Conteneurs
NAME        STATE     STATUS
jenkins     running   Up 3 minutes
sonar-db    running   Up 4 hours (healthy)
sonarqube   running   Up 4 hours

== Services
  [OK]   Jenkins répond (port 8081)          HTTP 200
  [OK]   SonarQube status UP (port 9000)      "status":"UP"
  [OK]   Jenkins -> SonarQube (réseau ci)     HTTP 200

== Outils dans le conteneur Jenkins
  [OK]   Node.js >= 22                        v22.23.4
  [OK]   npm                                  10.9.9
  [OK]   Docker CLI -> daemon de la VM        Server 29.9.2
  [OK]   Docker buildx                        github.com/docker/buildx v0.38.1 0dbe87f36ed472fe36b6796c3f70b4210a731fc
5
  [OK]   git                                  git version 2.47.3

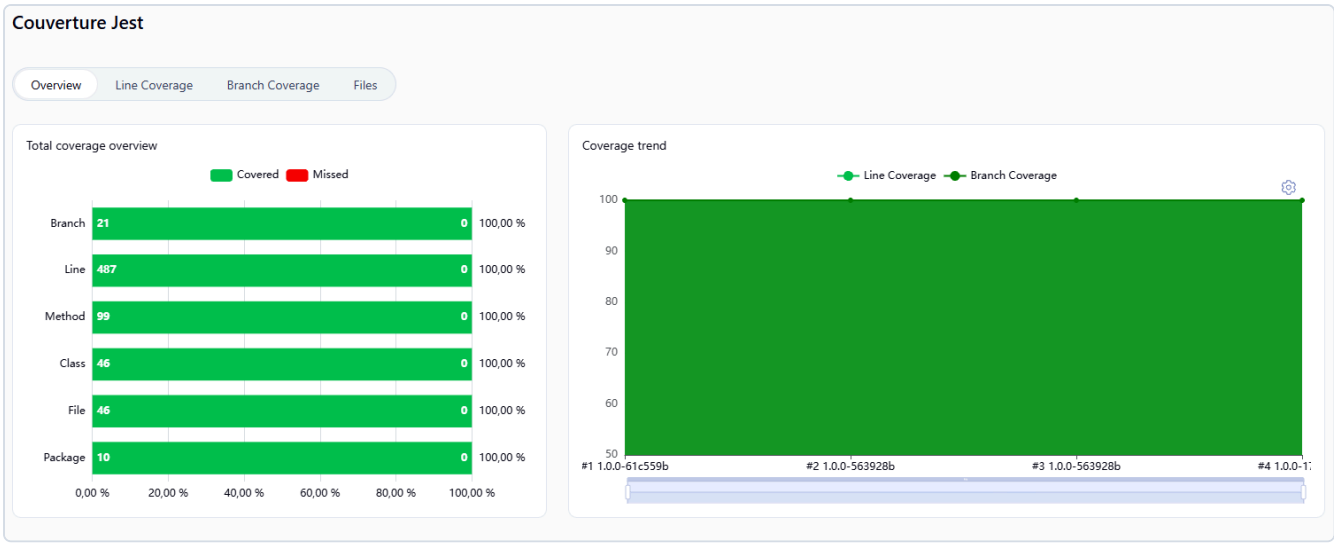
Lab opérationnel.
```

`check-lab.sh` : lab opérationnel

3. Tests et couverture

Le dépôt fourni ne contenait aucun test. J'ai ajouté **97 tests** (Jest + Supertest) qui couvrent l'authentification, les opérations sur les candidats, offres, candidatures et utilisateurs, ainsi que le contrôle d'accès par rôle. La base de données est simulée, les tests tournent donc sans PostgreSQL.

La couverture atteint **100 %**. Le pipeline échoue si elle passe sous 80 %.

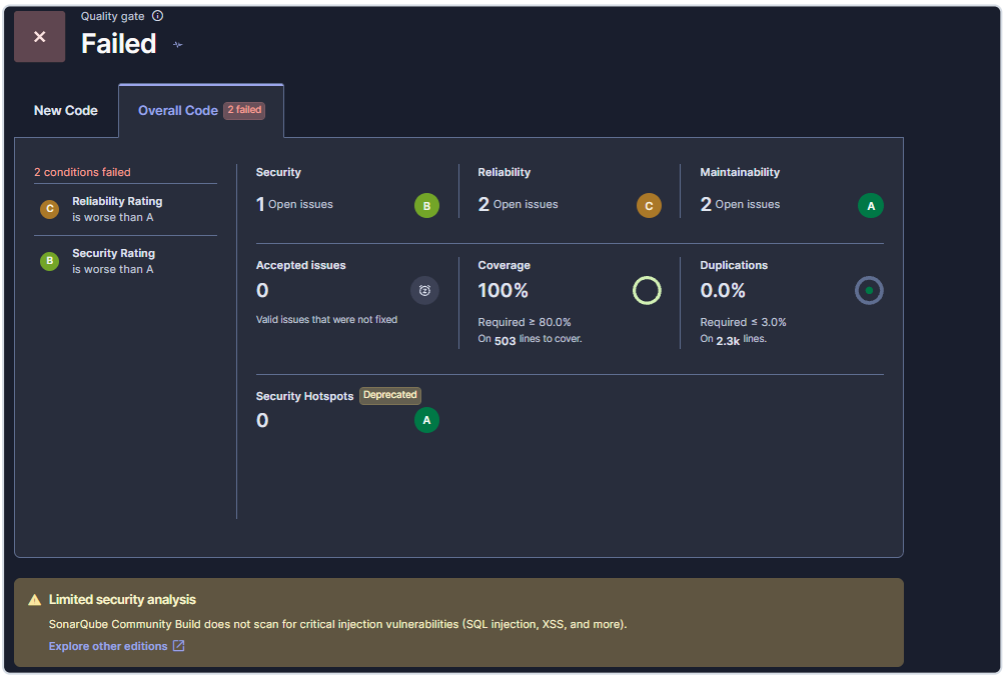


Couverture publiée dans Jenkins à chaque build

4. SonarQube et Quality Gate

Le code est analysé par SonarQube à chaque build, et Jenkins attend le verdict du Quality Gate avant de continuer. Le gate par défaut ne contrôlant que le nouveau code, j'ai créé un gate *SouverainRecrut* qui porte sur tout le code : couverture ≥ 80 %, duplications ≤ 3 %, note A en fiabilité, sécurité et maintenabilité, et tous les Security Hotspots revus.

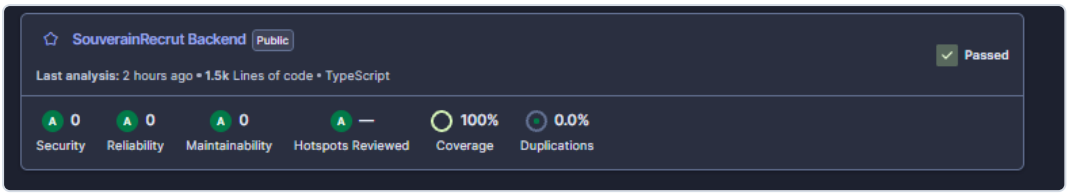
Avec ce gate, le **build #3 a été bloqué** par trois problèmes que le gate par défaut laissait passer :



Build #3 : Quality Gate Failed

- **Sécurité :** Express affichait sa version dans l'en-tête `X-Powered-By`, désactivé avec `app.disable("x-powered-by")`.
- **Fiabilité :** deux `parseInt` remplacés par `Number.parseInt` dans la pagination.

Au build #4, le gate passe avec la note A partout et aucun problème restant.



Build #4 : Quality Gate Passed

5. Image Docker et Trivy

Image Docker

Le Dockerfile fourni a été revu : Node 22 Alpine, build en deux étapes (l'image finale ne contient que le code compilé et les dépendances de production), exécution sans droits root, `.dockerignore`, healthcheck, et le commit source inscrit dans l'image. Chaque image reçoit deux tags : `<version>-<commit>` (ex. `1.0.0-8b039e7`), qui ne change jamais et sert au déploiement, et `dev`, qui désigne la dernière image validée.

Scan Trivy

Trivy scanne l'image après sa construction. Le pipeline s'arrête si une vulnérabilité **CRITICAL** a un correctif disponible ; les autres sont seulement rapportées.

Le **build #1 a été bloqué** par une CVE critique dans `proxy-addr`, une dépendance d'Express. Une mise à jour du lockfile (`npm update proxy-addr`) a suffi, et le build #2 a été publié.

```
14:35:35  Node.js (node-pkg)
14:35:35  ==================
14:35:35  Total: 1 (CRITICAL: 1)
14:35:35
14:35:35  ┌──────────────────────────┬─────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────────────────────────┐
14:35:35  │         Library          │ Vulnerability   │ Severity │ Status │ Installed Version │ Fixed Version │                          Title                           │
14:35:35  ├──────────────────────────┼─────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────┤
14:35:35  │ proxy-addr (package.json) │ CVE-2026-90711 │ CRITICAL │ fixed  │ 2.0.7             │ 2.0.8         │ proxy-addr: proxy-addr vulnerable to IP spoofing via     │
14:35:35  │                          │                 │          │        │                   │               │ IPv4-mapped IPv6 trust subnet                            │
14:35:35  │                          │                 │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-90711               │
14:35:35  └──────────────────────────┴─────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────────────────────────┘
```

Build #1 : CVE critique détectée par Trivy

J'ai ensuite réduit les vulnérabilités HIGH de l'image, en retirant npm de l'image finale et en mettant à jour des dépendances. L'image durcie a été publiée par le build #5 :

	CRITICAL	HIGH	MEDIUM	LOW
Build #4 (`1.0.0-172892d`)	0	26	20	3
Build #5 (`1.0.0-8b039e7`)	0	**1**	4	0

La vulnérabilité HIGH restante concerne un outil interne de Prisma, utilisé uniquement pendant les migrations ; elle sera corrigée avec Prisma 7.

6. Pipeline Jenkins

Le pipeline est décrit dans `integration-continue/Jenkinsfile` et s'exécute sur la branche `dev` :

Checkout → npm ci → prisma generate → Compilation TS → Tests + couverture → Analyse SonarQube → Quality Gate → Build image → Scan Trivy → **Publication Docker Hub**

Toute étape en échec arrête le pipeline : la publication, qui est la dernière étape, n'a lieu que si tous les contrôles sont au vert. Les secrets (token SonarQube, access token Docker Hub) sont stockés dans les credentials Jenkins, jamais dans le dépôt ni dans l'image.

```
15:52:26  Image publiée : <compte>/souverainrecrut-backend:1.0.0-172892d (+ tag dev)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] archiveArtifacts
15:52:26  Archiving artifacts
[Pipeline] sh
15:52:26  + docker image ls --format {{.Repository}}:{{.Tag}}
```

Fin d'un build : image publiée avec ses deux tags

7. Difficultés rencontrées

Difficulté	Solution
Port 8080 déjà utilisé sur le VPS	Jenkins déplacé sur le port 8081
Docker contourne le pare-feu pour les ports publiés	Écoute sur 127.0.0.1 et accès par tunnel SSH
Webhook SonarQube → Jenkins refusé (adresse privée)	Validation des webhooks locaux désactivée dans SonarQube
Couverture des branches à 70 % au premier essai	Tests complémentaires sur la pagination → 100 %